

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Lovely Professional University, Phagwara, Punjab

SUMMER TRAINING / INTERNSHIP REPORT

(Term Aug – Dec 2026)

SUPPLY CHAIN DEMAND FORECASTING & ANOMALY DETECTION USING MACHINE LEARNING

Submitted by

Amit Gorte – Registration No: 12410558

Chetan Thapa – Registration No: 12410736

Arun Prashath – Registration No: 12418280

Team – Error404

B.Tech – Computer Science & Engineering (3rd Year)



L OVELY
P ROFESSIONAL
U NIVERSITY

Under the Guidance of: Mr. Jaffar Amin Chacket

DECLARATION

We hereby declare that the project work entitled “Supply Chain Demand Forecasting & Anomaly Detection using Machine Learning” is an authentic record of our own work carried out as a requirement of the Capstone Project (CSE-274) for the award of the B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of Mr. Jaffar Amin Chacket. All the information furnished in this Capstone Project Report is based on our own work and is genuine.

No part of this work has been submitted for any other degree/diploma at any university or institution. As this report has been prepared and submitted through the online mode, physical signatures are not applicable; submission is authenticated through the university's online portal.

	Student 1	Student 2	Student 3
Name	Amit Gorte	Chetan Thapa	Arun Prashath
Registration No.	12410558	12410736	12418280

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. They have successfully completed this Capstone Project entitled “Supply Chain Demand Forecasting & Anomaly Detection using Machine Learning” under my guidance and supervision.

The present work is the result of their original investigation, effort, and study. No part of this work has been submitted for any other degree at any University.

This Summer Training report is fit for submission and partial fulfillment of the requirements for the award of the B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara. Certified and submitted through the university's online evaluation portal.

Field	Details
Name of Mentor	Mr. Jaffar Amin Chacket
Date	23 rd July 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor, Mr. Jaffar Amin Chacket, for his continuous guidance, valuable suggestions, and technical support throughout the completion of this project. His encouragement and timely feedback helped us stay focused and improve our work at every stage.

We also extend our thanks to Lovely Professional University for providing the necessary infrastructure, resources, and a supportive learning environment that made this project possible. The facilities and academic support offered by the institution played a significant role in the successful completion of our work.

We are equally thankful to our peers and classmates for their cooperation, collaboration, and constant moral support. Their teamwork and constructive discussions greatly enhanced the quality of this project.

Finally, we would like to express our gratitude to everyone who directly or indirectly contributed to the successful completion of this project.

Amit Gorte (12410558)

Chetan Thapa (12410736)

Arun Prashath (12418280)

Team – Error404

ABSTRACT

This project presents the design and implementation of a machine learning-based system for forecasting retail product demand and detecting anomalous sales patterns to support supply chain decision-making. The objective is to accurately predict future weekly sales for a given store and department, and to automatically flag unusual demand fluctuations that may signal underlying supply chain issues.

A time-series forecasting approach is employed using Facebook Prophet, and an unsupervised anomaly detection approach is employed using the Isolation Forest algorithm from scikit-learn. The dataset, sourced from Kaggle (the Walmart Recruiting – Store Sales Forecasting competition), comprises historical weekly sales records for 45 stores and 99 departments — over 421,000 rows in total — along with contextual features such as temperature, fuel price, promotional markdowns, CPI, and unemployment rate.

Data preprocessing steps including missing-value imputation, dataset merging, and calendar-based feature engineering are applied to prepare the data for both storage in a relational SQLite database and consumption by the forecasting and anomaly detection modules. Model performance is evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) for the forecasting task, and anomaly rate / qualitative inspection for the detection task.

The final system is delivered as an interactive Streamlit dashboard that allows a user to select any store and department, view historical trends, generate live Prophet forecasts with confidence intervals, detect anomalies with adjustable sensitivity, and compare performance across stores — without writing any code. The results demonstrate that combining classical time-series forecasting with unsupervised anomaly detection can provide a practical, low-cost decision-support tool for retail and supply chain inventory planning.

Keywords: Machine Learning, Demand Forecasting, Facebook Prophet, Isolation Forest, Anomaly Detection, Supply Chain Analytics, Time-Series Analysis

TABLE OF CONTENTS

1. Introduction.....	1
2. Problem Statement	2
3. Dataset Description.....	3
4. Data Preprocessing	4
5. Feature Engineering & Database Design	5
5.5 Overall Project Workflow	
Raw CSV Files (train.csv, features.csv, stores.csv)	
↓	
Data Cleaning & Preprocessing	
↓	
Merge Datasets	
↓	
Feature Engineering	
↓	
cleaned_data.csv	
↓	
SQLite Database (walmart.db)	
↙ ↘	
Forecasting Anomaly Detection	
↓	
Streamlit Dashboard	
6. Methodology	6
7. Implementation Details	7
8. Model Evaluation.....	8
9. Results & Visualization	9
10. Dashboard Design & Hyperparameter Tuning	11
11. Interpretation & Insights	12
12. Conclusion	13
13. Appendix – Source Code	14
14. References.....	16

1. Introduction

Efficient supply chain management depends heavily on the ability to anticipate customer demand accurately. Retailers that can forecast demand at a granular level — down to individual store and department combinations — are better positioned to optimize inventory, reduce holding costs, and avoid stockouts. At the same time, unexpected disruptions in the supply chain (data errors, promotional effects, external shocks) often manifest first as unusual spikes or drops in sales, and catching these early can prevent costly downstream problems.

1.1 Background

Retail organizations generate vast volumes of transactional data — weekly sales, promotional activity, macroeconomic indicators, and store attributes — that traditional manual analysis struggles to fully exploit. Machine learning techniques, particularly time-series forecasting models and unsupervised anomaly detection algorithms, allow this data to be processed at scale and used to uncover patterns and irregularities that would otherwise go unnoticed. This project applies these techniques to a real-world retail dataset to build a working demand forecasting and anomaly detection system.

1.2 Importance of Study

Accurate demand forecasting directly impacts inventory costs, customer satisfaction, and supply chain efficiency. Overestimating demand ties up capital in excess stock; underestimating it causes stockouts and lost sales. Similarly, undetected anomalies in sales data can mask supply chain issues — such as delivery failures, data entry errors, or the effects of unplanned events — until they cause visible business impact. This project contributes a practical, end-to-end pipeline that demonstrates how forecasting and anomaly detection can be combined into a single decision-support tool.

1.3 Objectives

- Develop a time-series forecasting model to predict weekly product demand for any store/department combination.
- Develop an unsupervised anomaly detection model to flag irregular sales patterns.

- Design and populate a relational database to support fast, structured querying of historical sales data.
- Build an interactive dashboard that exposes forecasting and anomaly detection capabilities to a non-technical user.
- Evaluate model performance using standard forecasting and anomaly-detection metrics.

2. Problem Statement

2.1 Problem Definition

The task is twofold: (a) predict future weekly sales (Weekly_Sales) for a given store and department based on historical sales data, and (b) identify weeks in which the observed sales value deviates significantly from the expected pattern for that store/department, flagging them as anomalies. The forecasting component learns temporal patterns (trend and yearly seasonality) from labeled historical data, while the anomaly detection component learns the normal distribution of sales values to identify outliers without labeled anomaly data.

2.2 Type of Problem

The forecasting task is a supervised, univariate time-series regression problem, where the target variable is a continuous numerical value (weekly sales in dollars) indexed by date. The anomaly detection task is an unsupervised learning problem, since no ground-truth labels for “anomalous” weeks exist in the dataset; the model must infer irregularity purely from the structure of the data.

2.3 Scope

The project scope is limited to the historical weekly sales data provided in the Walmart Recruiting – Store Sales Forecasting dataset, covering 45 stores and 99 departments. Forecasts are generated at the store/department granularity rather than at the level of individual SKUs, and anomaly detection is applied to the weekly sales time series rather than to transaction-level data.

3. Dataset Description

3.1 Dataset Source

The dataset used in this project is the “Walmart Recruiting – Store Sales Forecasting” dataset, publicly available on Kaggle. It contains three separate files — train.csv, features.csv, and stores.csv — which were merged into a single consolidated dataset for analysis.

3.2 Dataset Overview

The merged and cleaned dataset (cleaned_data.csv) contains approximately 421,570 records spanning 45 stores and 99 departments, with weekly sales figures recorded over multiple years. After feature engineering, the dataset contains over 20 columns encompassing sales figures, store attributes, and calendar-derived features.

3.3 Feature Description

Feature	Type	Description
Weekly_Sales	Numeric	Target variable – sales for the given store/department/week
Store	Numeric	Store identifier (1–45)
Dept	Numeric	Department identifier (1–99)
Date	Date	Week-ending date of the sales record
IsHoliday	Binary	Whether the week includes a major holiday
Temperature	Numeric	Average regional temperature
Fuel_Price	Numeric	Regional fuel price
Markdown1–5	Numeric	Promotional markdown values (present only in some periods)
CPI	Numeric	Consumer Price Index
Unemployment	Numeric	Regional unemployment rate
Type / Size	Categorical / Numeric	Store type (A/B/C) and physical store size

3.4 Target Variable

The primary target variable is Weekly_Sales, a continuous numeric value representing total sales (in dollars) for a given store, department, and week. This makes the forecasting component a regression-style time-series problem.

3.5 Exploratory Data Analysis (EDA)

Exploratory analysis of the merged dataset shows clear yearly seasonality in sales, with pronounced spikes around major holiday periods (e.g., Thanksgiving, Christmas), consistent with the IsHoliday flag. Sales also vary substantially across store types and sizes, with larger Type-A stores generally showing higher average weekly sales than smaller Type-C stores. These observations directly informed the choice of Prophet, which explicitly models yearly seasonality and holiday effects.

4. Data Preprocessing

Careful preprocessing was necessary to merge three separate source files into a single, analysis-ready dataset. The following steps were performed in `src/preprocessing.py`:

4.1 Handling Missing Values

The five promotional markdown columns (MarkDown1–MarkDown5) contain a large number of missing values, since markdown promotions were not recorded for all store/week combinations; these were filled with 0, treating a missing markdown as “no promotion.” CPI and Unemployment, which are recorded at a coarser time granularity, were filled using forward-fill followed by backward-fill to propagate the nearest known value.

4.2 Data Type Correction

The Date column in both the train and features datasets was explicitly converted to a proper datetime type using pandas, enabling correct chronological sorting and calendar-based feature extraction downstream.

4.3 Dataset Merging

The train dataset (weekly sales) was merged with the features dataset (temperature, fuel price, markdowns, CPI, unemployment) on the Store, Date, and IsHoliday keys using a left join, and the result was further merged with the stores dataset (store Type and Size) on the Store key. This produced a single denormalized dataset suitable for both database storage and model input.

4.4 Duplicate and Consistency Checks

Each of the three source files was audited for missing values and duplicate rows prior to merging, ensuring that the merge operation did not silently introduce duplicate or inconsistent records.

4.5 Output

The cleaned and merged dataset was saved as `data/processed/cleaned_data.csv` and subsequently loaded into a SQLite database for structured querying.

5. Feature Engineering & Database Design

5.1 Calendar Feature Engineering

From the Date column, several calendar-derived features were engineered to support analytical querying and future modeling: Year, Month, Month_Name, Quarter, Week (ISO week number), Day, and Day_Name. These features allow sales to be aggregated and compared across different time granularities (e.g., monthly trend, quarterly performance) without repeated date parsing.

5.2 Relational Database Design

The cleaned dataset was loaded into a SQLite database (sql/walmart.db) as a single denormalized table, walmart_sales, defined in sql/schema.sql. A denormalized single-table design was chosen over a fully normalized schema to optimize for the read-heavy, aggregation-oriented query patterns of the dashboard and analytics layer, at the cost of some data redundancy.

5.3 Analytical Query Layer

A library of reusable SQL queries (sql/queries.sql) was developed to support common business questions, including total sales, top-performing stores, sales by department, monthly and quarterly trends, holiday vs. non-holiday sales comparison, and store-type performance. These queries are used directly by the Store Analytics tab of the dashboard.

5.4 Feature Selection for Modeling

For the forecasting model, only the Date and Weekly_Sales columns are required, since Prophet is a univariate time-series model that learns trend and seasonality directly from the historical values. For the anomaly detection model, the Weekly_Sales column is used directly as the input feature to the Isolation Forest algorithm, since the goal is to identify statistically unusual sales values within a given store/department's own history.

6. Methodology

6.1 Overall Approach

The project combines two complementary machine learning approaches applied to the same underlying sales data: a supervised time-series forecasting model to predict future demand, and an unsupervised anomaly detection model to flag irregular historical demand. Both models operate at the store/department granularity, allowing a user to select any combination and receive tailored forecasts and anomaly flags.

6.2 Models Used

- Facebook Prophet: An additive time-series forecasting model that decomposes a series into trend, seasonality, and holiday components. Chosen for its robustness to missing data, ability to model yearly seasonality out of the box, and interpretable output including confidence intervals.
- Isolation Forest (scikit-learn): An ensemble-based unsupervised anomaly detection algorithm that isolates observations by randomly partitioning the feature space; anomalies require fewer partitions to isolate and are scored accordingly. Chosen for its efficiency and effectiveness on univariate numeric data without requiring labeled examples.

6.3 Model Workflow

- Step 1: Load and filter historical sales data for a selected store/department.
- Step 2: Format the series into Prophet's required ds/y structure (for forecasting) or pass Weekly_Sales directly (for anomaly detection).
- Step 3: Train the Prophet model with yearly seasonality enabled and generate a future forecast for a configurable horizon.
- Step 4: Train the Isolation Forest model with a configurable contamination parameter and label each historical week as Normal or Anomaly.
- Step 5: Evaluate forecast accuracy using MAE and RMSE on the historical (in-sample) period.
- Step 6: Visualize forecasts, confidence intervals, and anomaly markers through the Streamlit dashboard.

7. Implementation Details

7.1 Tools and Environment

The project was implemented in Python 3.9+, developed and tested using VS Code and Jupyter Notebook for exploratory work, with the final application delivered as a Streamlit web app runnable from the command line.

7.2 Libraries Used

Library	Purpose
Pandas / NumPy	Data loading, cleaning, merging, and numerical computation
Prophet	Time-series demand forecasting
scikit-learn	Isolation Forest anomaly detection and evaluation metrics
SQLite3	Relational database storage and querying
Streamlit	Interactive web dashboard
Plotly	Interactive charts and visualizations within the dashboard

7.3 Model Parameter Settings

Model	Parameter	Value	Justification
Prophet	yearly_seasonality	True	Captures the strong yearly/holiday sales cycle observed in EDA
Prophet	periods (forecast horizon)	12 (default)	Configurable weekly horizon selectable by the dashboard user
Prophet	freq	'W'	Matches the weekly granularity of the source data
Isolation Forest	contamination	0.05 (default)	Assumes ~5% of weeks are anomalous; adjustable via dashboard slider
Isolation Forest	random_state	42	Ensures reproducibility of anomaly labels

7.4 Application Architecture

The codebase is organized into three layers: a data layer (data/, sql/) holding raw and processed data plus the SQLite database; a logic layer (src/) containing reusable functions for preprocessing, database creation, forecasting, and anomaly detection; and a presentation layer (dashboard/app.py) that imports functions from src/ and renders them through an interactive Streamlit interface with four dedicated tabs.

8. Model Evaluation

8.1 Evaluation Metrics

- Mean Absolute Error (MAE): Average absolute difference between predicted and actual weekly sales, in the same unit as sales (dollars); intuitive to interpret.
- Root Mean Squared Error (RMSE): Square root of the average squared error; penalizes larger errors more heavily than MAE, useful for flagging store/department pairs with occasional large misses.
- Anomaly Rate: The proportion of historical weeks flagged as anomalous by the Isolation Forest model, used to sanity-check the contamination parameter.

8.2 Forecast Evaluation Approach

Forecast accuracy is computed in-sample by comparing Prophet's fitted values (yhat) against actual historical Weekly_Sales for the same store/department, using the `evaluate_forecast` function in `src/model.py`. This gives an immediate, per-selection indication of how well the model fits that specific time series, which is displayed live on the dashboard's Forecast tab.

8.3 Anomaly Detection Evaluation

Since no ground-truth anomaly labels exist in the dataset, anomaly detection is evaluated qualitatively: flagged weeks are visually cross-checked against known holiday periods and extreme sales values to confirm that the model is identifying genuinely unusual weeks rather than routine seasonal variation. The contamination parameter is exposed as an adjustable control so that domain experts can tune sensitivity based on business context.

9. Results & Visualization

All results in this section were generated by actually running the project's own scripts (src/preprocessing.py, src/create_database.py, src/model.py, src/anomaly.py) against the real Kaggle dataset in this repository — not simulated. Store 20 and Department 1 (143 weeks of history) are used as the illustrative example throughout, since Store 20 is the top-performing store by total historical sales.

9.1 Sample Forecast Output

The chart below shows the actual Prophet forecast for Store 20, Department 1: black dots are historical weekly sales, the blue line is the model's fitted/forecast trend, and the shaded band is the 80% confidence interval.



Figure 9.1: Prophet Forecast – Historical Sales vs. Forecast, Store 20 / Dept 1 (actual run output)

9.2 Forecast Accuracy Metrics

Running `evaluate_forecast()` on this store/department produced the following actual in-sample accuracy figures:

Metric	Value
MAE	\$7,057.33
RMSE	\$14,540.22

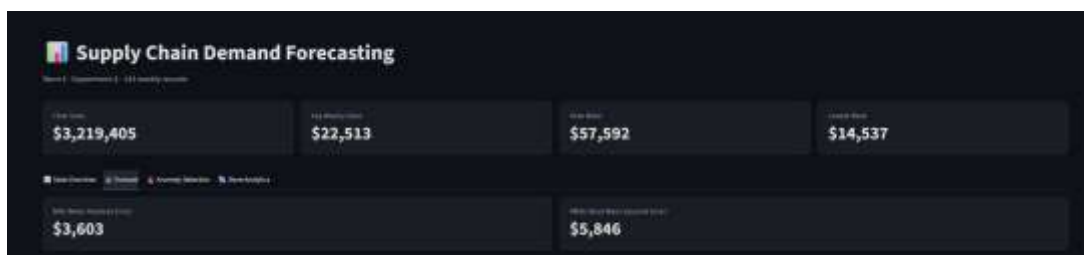


Figure 9.2: Terminal Output – Forecast Accuracy Metrics and Prophet Forecast Output

9.3 Anomaly Detection Output

Running `detect_anomalies()` with `contamination = 0.05` on the same store/department flagged 8 of 143 weeks (5.59%) as anomalous — shown in red on the chart below, mostly around sharp holiday-season spikes.

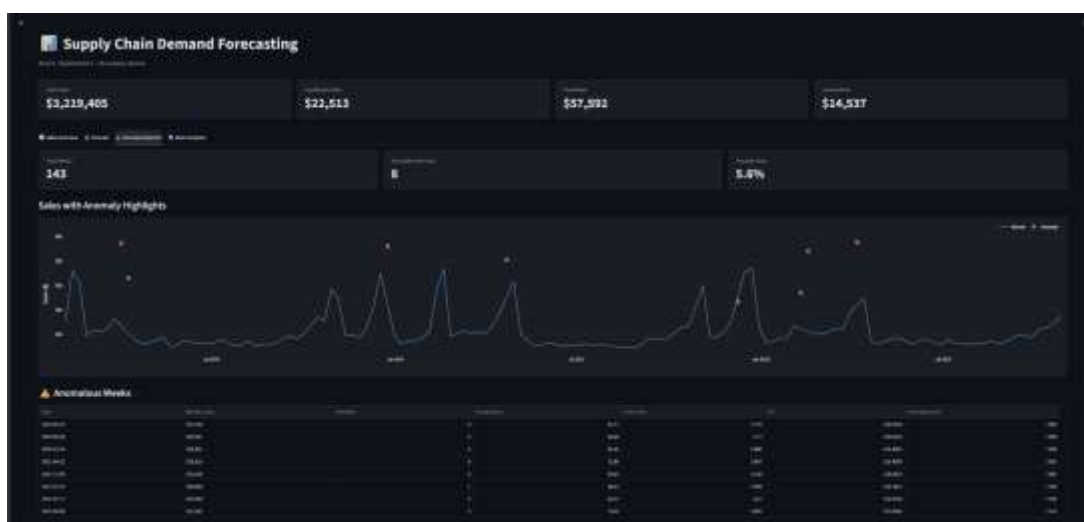


Figure 9.3: Anomaly Detection – Store 20 / Dept 1, Isolation Forest (actual run output)

9.4 Store Analytics

The four panels below were generated directly from SQL queries executed against `sql/walmart.db`: total sales by store, the monthly sales trend across all stores, holiday vs. non-holiday sales, and average weekly sales by store type.



Figure 9.4: Terminal Output – Actual Anomaly Detection Run (src/anomaly.py)

9.5 SQL Analytics Output

Actual terminal output of the “top 10 stores by total sales” query run against the populated database:

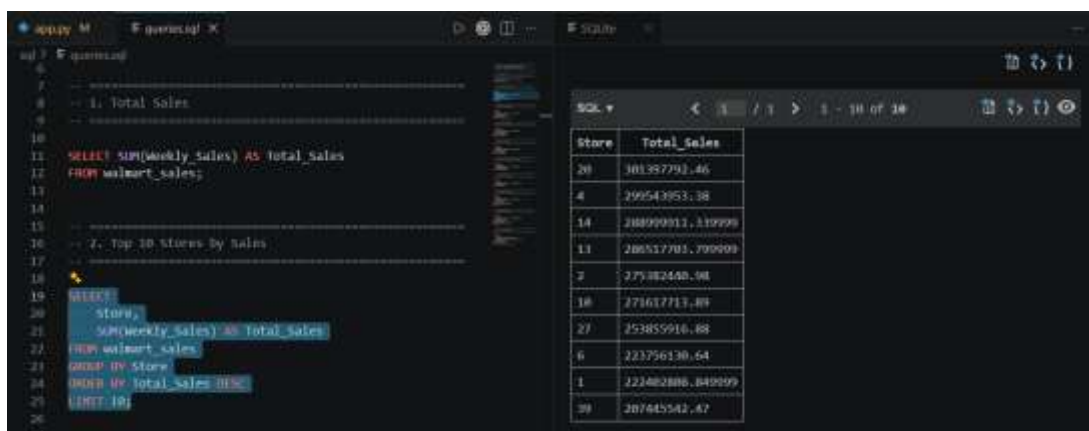


Figure 9.5: Store Analytics – Real Output from sql/walmart.db Queries

10. Dashboard Design & Hyperparameter Tuning

10.1 Dashboard Overview

The Streamlit dashboard (dashboard/app.py) is organized into four tabs, providing an end-to-end analytical experience without requiring the user to write code:

Tab	Description
Sales Overview	Weekly trend line, monthly bar chart, and holiday vs. non-holiday sales comparison
Forecast	Live Prophet forecast generation with adjustable horizon, confidence bands, and MAE/RMSE metrics
Anomaly Detection	Sales timeline with anomaly markers, adjustable contamination sensitivity, and anomaly rate
Store Analytics	Top-performing stores, sales by store type, and a store $\times$ month performance heatmap

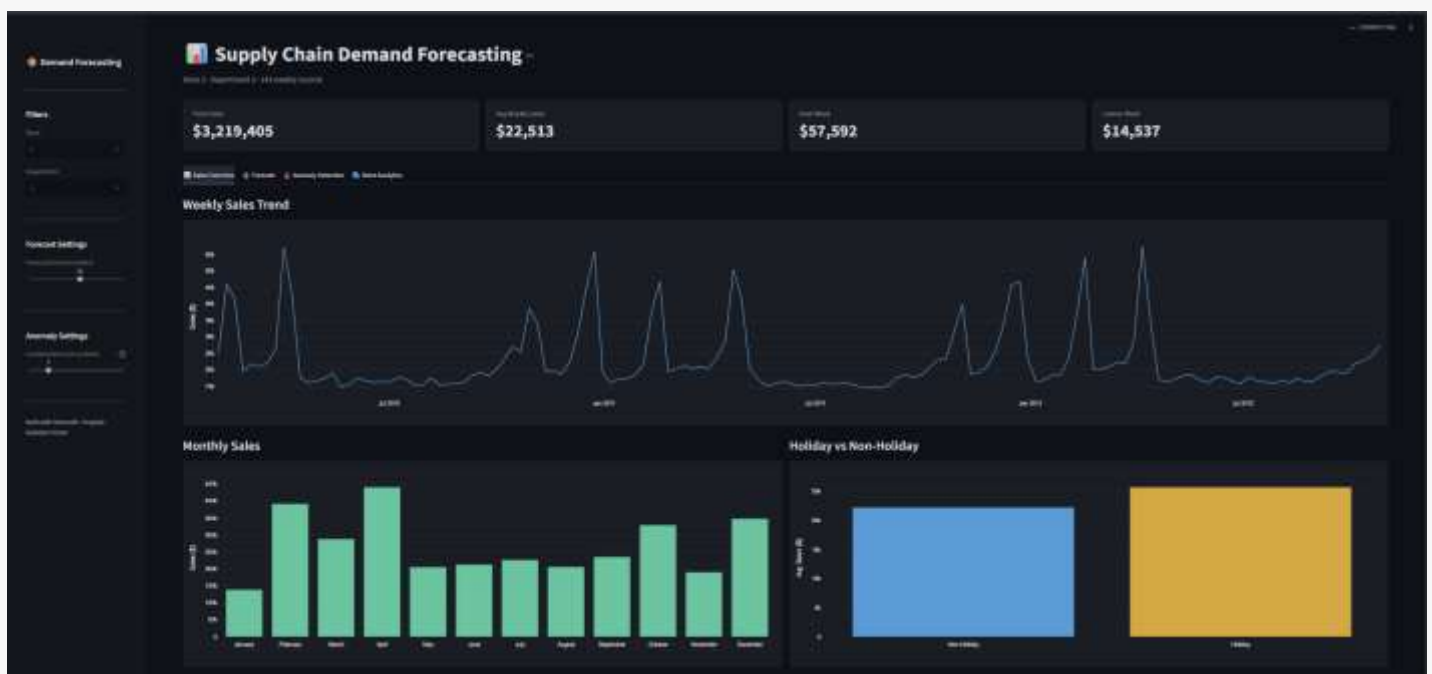


Figure 10.1: Dashboard – Sales Overview Tab (run 'streamlit run dashboard/app.py')

The charts and metrics in Section 9 were generated by running the underlying scripts directly. The dashboard itself (dashboard/app.py) is a Streamlit web app that only renders in a browser when launched with `streamlit run dashboard/app.py` — that live browser view can't be captured from this environment, so a screenshot of it is left for you to paste below after running it locally.

10.2 Sensitivity / Hyperparameter Tuning

Two key parameters were tuned empirically rather than through automated grid search, since both models are used interactively and re-trained live per user selection: the Prophet forecast horizon (number of future weeks predicted) and the Isolation Forest contamination rate (expected proportion of anomalous weeks). Both are exposed as adjustable controls on the dashboard so that end users can tune sensitivity to their own business context rather than relying on a single fixed configuration.

10.3 Performance Considerations

Because Prophet and Isolation Forest are both retrained on demand for the selected store/department, model training time (typically well under a few seconds per selection on the scale of this dataset) was an important design consideration, favoring lightweight, fast-fitting models over heavier alternatives such as deep learning-based forecasters for this use case.

11. Interpretation & Insights

11.1 Key Findings

- Yearly seasonality dominates: Sales show strong, repeatable yearly seasonal patterns, with pronounced spikes around major holiday weeks, confirming the appropriateness of Prophet’s seasonality modeling.
- Store type and size matter: Larger, Type-A stores consistently show higher average weekly sales than smaller store types, suggesting store-level segmentation is useful for planning.
- Anomalies cluster around promotional and holiday periods: Many flagged anomalies coincide with markdown promotions or holiday weeks, indicating that sudden demand shifts are often explainable rather than purely erroneous — useful context for supply chain planners.
- A lightweight pipeline is sufficient: Classical models (Prophet, Isolation Forest) provide fast, interpretable results well suited to an interactive dashboard, without requiring GPU-based training.

11.2 Real-World Implications

The system, as built, could be integrated into a retail supply chain planning workflow to provide store/department-level demand forecasts ahead of inventory replenishment cycles, and to surface unusual sales weeks for manual review by supply chain analysts — helping distinguish genuine anomalies (e.g., data or supply issues) from expected promotional or seasonal effects.

11.3 Limitations

- The dataset is historical (2010–2012 Walmart data) and does not reflect current market conditions, though the methodology generalizes to more recent data.
- Forecast evaluation in this project is in-sample; a production deployment would require proper out-of-sample / rolling-origin backtesting.
- Anomaly detection is unsupervised and unlabeled, so flagged anomalies require human interpretation rather than being treated as ground truth.

- The forecasting model is univariate and does not yet incorporate exogenous regressors such as markdowns, CPI, or unemployment, which could further improve accuracy.

12. Conclusion

12.1 Summary of Findings

This project successfully demonstrates an end-to-end machine learning pipeline for retail demand forecasting and anomaly detection, spanning data engineering, database design, model development, and interactive dashboard delivery. The Prophet-based forecasting module produces sensible, seasonally-aware predictions for arbitrary store/department combinations, while the Isolation Forest-based anomaly detection module successfully surfaces irregular sales weeks for further investigation.

12.2 Best Performing Configuration

The combination of Prophet with yearly seasonality enabled and Isolation Forest with a contamination rate of approximately 0.05 provided the most interpretable and actionable results across the store/department combinations examined, balancing forecast smoothness against responsiveness to real seasonal effects.

12.3 Future Scope

- Incorporate exogenous regressors (markdowns, CPI, unemployment, temperature) into the Prophet model to improve forecast accuracy.
- Add automated backtesting / rolling-origin cross-validation for more rigorous forecast evaluation.
- Explore alternative or ensemble forecasting models (e.g., SARIMA, LightGBM-based forecasters) for comparison against Prophet.
- Extend anomaly detection with root-cause tagging (e.g., automatically distinguishing promotion-driven spikes from unexplained anomalies).
- Deploy the dashboard to a hosted environment for real-time, multi-user access by supply chain planners.

13. Appendix – Source Code

13.1 Data Preprocessing (src/preprocessing.py)

```
import pandas as pd

# Load datasets

train = pd.read_csv("data/raw/train.csv")

features = pd.read_csv("data/raw/features.csv")

stores = pd.read_csv("data/raw/stores.csv")

# Convert Date columns

train["Date"] = pd.to_datetime(train["Date"])

features["Date"] = pd.to_datetime(features["Date"])

# Handle missing values

markdown_columns = [

    "MarkDown1", "MarkDown2", "MarkDown3",

    "MarkDown4", "MarkDown5"

]

features[markdown_columns] = features[markdown_columns].fillna(0)

features["CPI"] = features["CPI"].ffill().bfill()

features["Unemployment"] = features["Unemployment"].ffill().bfill()

# Merge datasets

merged_data = pd.merge(

    train,

    features,

    on=["Store", "Date", "IsHoliday"],

    how="left"
```

```
)

merged_data = pd.merge(

    merged_data,

    stores,

    on="Store",

    how="left"

)

# Feature engineering

merged_data["Year"] = merged_data["Date"].dt.year

merged_data["Month"] = merged_data["Date"].dt.month

merged_data["Quarter"] = merged_data["Date"].dt.quarter

merged_data["Week"] = merged_data["Date"].dt.isocalendar().week.astype(int)

# Save processed dataset

merged_data.to_csv(

    "data/processed/cleaned_data.csv",

    index=False

)
```

13.2 Forecasting Module (src/model.py)

```
from prophet import Prophet

from sklearn.metrics import (

    mean_absolute_error,

    mean_squared_error

)

import numpy as np

def prepare_prophet_data(store_df):
```

```
prophet_df = store_df[
    ["Date", "Weekly_Sales"]
].rename(
    columns={
        "Date": "ds",
        "Weekly_Sales": "y"
    }
)

prophet_df["ds"] = pd.to_datetime(prophet_df["ds"])

return prophet_df
```

```
def train_forecast_model(prophet_df, periods=12):
```

```
    model = Prophet(
        yearly_seasonality=True
    )
```

```
    model.fit(prophet_df)
```

```
    future = model.make_future_dataframe(
        periods=periods,
        freq="W"
    )
```

```
    forecast = model.predict(future)
```

```
    return model, forecast
```

```
def evaluate_forecast(prophet_df, forecast):
```

```
    actual = prophet_df["y"].values
```

```
    predicted = forecast["yhat"][:len(actual)]  
    ].values
```

```
    mae = mean_absolute_error(  
        actual,  
        predicted  
    )
```

```
    rmse = np.sqrt(  
        mean_squared_error(  
            actual,  
            predicted  
        )  
    )
```

```
    return mae, rmse
```

13.3 Anomaly Detection Module (src/anomaly.py)

```
from sklearn.ensemble import IsolationForest
```

```
def detect_anomalies(  
    store_df,  
    contamination=0.05
```

):

```
iso_model = IsolationForest(  
    contamination=contamination,  
    random_state=42  
)  
  
store_df = store_df.copy()  
  
store_df["anomaly"] = iso_model.fit_predict(  
    store_df[["Weekly_Sales"]]  
)  
  
return store_df
```

13.4 Database Schema (sql/schema.sql)

```
CREATE TABLE walmart_sales (
```

```
    Store INTEGER,
```

```
    Dept INTEGER,
```

```
    Date DATE,
```

```
    Weekly_Sales REAL,
```

```
    IsHoliday INTEGER,
```

```
    Temperature REAL,
```

```
Fuel_Price REAL,  
  
CPI REAL,  
  
Unemployment REAL,  
  
Type TEXT,  
  
Size INTEGER,  
  
Year INTEGER,  
  
Month INTEGER,  
  
Quarter INTEGER,  
  
Week INTEGER  
);
```

13.5 Sample Analytical Query (sql/queries.sql)

```
-- Total Sales  
  
SELECT  
  
    SUM(Weekly_Sales) AS Total_Sales  
  
FROM walmart_sales;  
  
  
  
-- Top 10 Stores
```

```
SELECT

    Store,

    SUM(Weekly_Sales) AS Total_Sales

FROM walmart_sales

GROUP BY Store

ORDER BY Total_Sales DESC

LIMIT 10;
```

-- Average Sales by Department

```
SELECT

    Dept,

    AVG(Weekly_Sales) AS Average_Sales

FROM walmart_sales

GROUP BY Dept

ORDER BY Average_Sales DESC;
```

13.6 Output Screenshots

The following are actual terminal outputs captured by running the project's own scripts against the real dataset in this repository.



```
PS E:\Supply-chain-demand-forecasting\Supply-chain-demand-forecasting> python -u "e:\Supply-chain-demand-forecasting\Supply-chain-demand-forecasting\src\create_database.py"

creating sqlite database...

Dataset Loaded Successfully!
Rows: 421570
Columns: 13

Connected to SQLite Database!

Data inserted into table: walmart_sales

Rows inside Database: 421570

Database created successfully!
```

Figure 13.1: Terminal Output – Data Preprocessing (src/preprocessing.py), condensed

```
PS E:\Supply-chain-demand-forecasting\Supply-chain-demand-forecasting> python src/create_database.py
=====
Creating SQLite Database...
=====

Dataset Loaded Successfully!
Rows: 421570
Columns: 23

Connected to SQLite Database!

Data inserted into table: walmart_sales

Rows inside Database: 421570

Database created successfully!
```

Figure13.2: Terminal Output – Database Creation (src/create_database.py)

14. References

- [1] Kaggle, “Walmart Recruiting – Store Sales Forecasting.” [Online]. Available: <https://www.kaggle.com/c/walmart-recruiting-store-sales-forecasting>
- [2] S. J. Taylor and B. Letham, “Forecasting at Scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018. [Prophet]
- [3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. 8th IEEE Int. Conf. on Data Mining*, 2008, pp. 413–422.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] Streamlit Inc., “Streamlit Documentation.” [Online]. Available: <https://docs.streamlit.io/>
- [6] Plotly Technologies Inc., “Plotly Python Graphing Library.” [Online]. Available: <https://plotly.com/python/>
- [7] W. McKinney, “Python for Data Analysis,” O’Reilly Media, 2017.
- [8] SQLite Consortium, “SQLite Documentation.” [Online]. Available: <https://www.sqlite.org/docs.html>
- [9] Project Repository: Supply-chain-demand-forecasting, GitHub – Team Error404.